# GRASP-RAG: Graph Retrieval with Adaptive Structured Paths and Path-Guided Reasoning

**[Author Name 1]**
**[Institution]**
**[City, Country]**
**[email@domain]**

**[Author Name 2]**
**[Institution]**
**[City, Country]**
**[email@domain]**

***Abstract*—Existing graph-based retrieval-augmented generation (RAG) systems apply a single, fixed retrieval strategy to every query regardless of its logical structure. We present GRASP-RAG (Graph Retrieval with Adaptive Structured Paths), which introduces two complementary innovations. First, *query-adaptive motif selection*: a lightweight LLM classifier identifies the logical shape of each query—single-entity, bridge, multi-entity, or broad—and selects the matching graph traversal motif: ego-network, bidirectional shortest path, Steiner-tree approximation, or Personalized PageRank diffusion. Second, *Path-Guided Reasoning* (PGR): the retrieved subgraph is linearized into a structured context that mirrors the graph's reasoning topology—a step-by-step hop chain for bridge queries, an entity profile for single-entity queries, and a comparison scaffold for multi-entity queries—so the LLM receives a reasoning scaffold that matches the answer shape. Evaluated on all four domains of UltraDomain (Agriculture, CS, Legal, Mix) using a typed question set stratified by query shape (500 questions per domain; 125 per type), GRASP-RAG achieves an 81.1% average win rate against LightRAG, 70.9% against NaiveRAG, and 40.8% against PathRAG under multi-judge LLM-as-judge evaluation (GPT-4o-mini, Claude Haiku, Gemini Flash; bias-corrected majority vote across five dimensions). The advantage is largest on bridge queries (93.4% vs. LightRAG, 84.4% vs. NaiveRAG) and single-entity queries (93.4% vs. LightRAG), confirming that matching the retrieval algorithm to the query's logical shape produces structurally better answers than fixed-strategy retrieval.***

***Index Terms*—retrieval-augmented generation, knowledge graph, graph RAG, multi-hop reasoning, query classification, path-guided reasoning**

## I. Introduction

Retrieval-Augmented Generation (RAG) [1] grounds large language models (LLMs) in external knowledge, reducing hallucination. Early RAG systems retrieve flat text chunks by vector similarity, losing the relational structure across entities essential for multi-hop reasoning.

Graph-based RAG systems [2]–[5] build a knowledge graph (KG) over the corpus and retrieve coherent subgraphs. Despite strong results, they share a critical design limitation: **every method applies one fixed retrieval strategy to all queries**, regardless of how the question is phrased or what kind of answer it calls for.

Natural language queries signal their own answer shape through their linguistic form [6]. *"What is X?"* asks for a definition—the answer is a self-contained profile of one entity, and the relevant context is the node's immediate neighbourhood. *"What connects X to Y?"* asks for a mechanism—the answer must name an intermediate entity and explain each link in the chain; the relevant context is a 2-hop path, not a neighbourhood [7]. *"How do X, Y, and Z interact?"* asks for synthesis across multiple entities [8]; the relevant context is the minimal subgraph that connects all three. *"What are the main themes of. . . ?"* asks for a thematic summary with no specific entity anchor; the relevant context is a diffusion over the graph from all plausible starting points. A system that ignores these linguistic cues and applies one fixed traversal to every query will inevitably retrieve mismatched context for most of them.

Consider two questions posed to the same agricultural KG: (1) *"What is Varroa Mite?"* and (2) *"What connects Rational Grazing to Polyface Farm?"* The verb *is* in (1) signals a definitional answer; *connects* in (2) signals a relational chain requiring an intermediate entity (Cows). PathRAG [5] applies the same path-retrieval strategy to both and answers (2) with fluent prose about "shared commitment to sustainable practices"—never naming Cows—because it never read the linguistic signal that (2) is a bridge problem requiring 2-hop traversal.

We present **GRASP-RAG**, which makes query shape a first-class concern. GRASP-RAG contributes:

1) **Query-adaptive motif selection**: a classifier maps each query to one of four graph motifs, selecting the subgraph shape that best matches the answer structure (Section III-E).
2) **Path-Guided Reasoning** (PGR): context is structured to mirror the graph's reasoning topology, not just a bag of retrieved passages (Section III-F).
3) **Candidate-based anchor linking**: entity VDB search supplies real graph node names to the classifier, eliminating fuzzy-match errors.

On a typed question set stratified by query shape across all four UltraDomain domains, GRASP-RAG achieves an 81.1% average win rate against LightRAG and 70.9% against NaiveRAG under multi-judge LLM-as-judge evaluation. The gain is largest on bridge and single-entity queries, where the structural match between motif and query type is most direct.

## II. Related Work

**LightRAG** [3] introduces dual-level retrieval: low-level (1-hop entity neighborhoods) and high-level (relation embeddings). Mode selection uses coarse keyword signals, not query-shape

classification; both modes apply fixed-radius expansion regardless of entity count.

**HippoRAG** [4] seeds Personalized PageRank (PPR) [9] from query entities. PPR naturally captures multi-hop associations but is applied uniformly to all query types—a single-entity definitional question receives the same global diffusion as a multi-hop integrative question.

**G-Retriever** [10] computes a prize-collecting Steiner tree spanning all query entities—effective for multi-entity questions but degenerating to a single node for single-entity queries.

**PathRAG** [5] is the closest prior work. Flow-based pruning extracts relational paths between anchor pairs, and a reliability-ordered path prompt exploits LLM recency bias. PathRAG's path format is the same for all query types. The key gap GRASP-RAG fills: *selecting the retrieval motif and context format conditional on the query's logical shape*.

**Layered Query Retrieval (LQR)** [11] is the closest work in spirit: it classifies each query by *complexity* (binary: simple vs. complex) and routes simple queries to a single BM25 retrieval step while decomposing complex queries into two sub-questions for iterative retrieval, followed by an NLI relevance filter. LQR shares GRASP-RAG's core motivation—query properties should govern retrieval strategy—but classifies at a coarser granularity and operates on flat text without a knowledge graph. Critically, LQR's binary split cannot distinguish between queries that require an ego-network (single-entity definitional) versus a 2-hop bridge traversal (relational chain)—both would be labelled "complex" and receive the same decomposition treatment. GRASP-RAG replaces binary complexity with a four-way *logical shape* taxonomy tied directly to graph traversal algorithms.

**BridgeRAG** [12] addresses the same failure mode as our bridge motif from a graph-free angle: it conditions second-hop candidate scoring on an intermediate bridge passage via a tripartite judge $s(q, b, c)$, achieving state-of-the-art training-free recall on HotpotQA, 2WikiMultiHopQA, and MuSiQue. Unlike GRASP-RAG, BridgeRAG has no knowledge graph and applies bridge conditioning uniformly to all queries; GRASP-RAG reserves bridge-path traversal for queries whose linguistic form signals a relational chain, routing other query shapes to structurally matched motifs.

## III. GRASP-RAG

### A. Overview

Figure 1 shows the complete GRASP-RAG system, covering both the query pipeline (top) and the multi-judge evaluation protocol (bottom). Figure 2 illustrates the four retrieval motifs in detail. At query time, (1) a VDB search retrieves candidate entity names from the graph; (2) an LLM classifier infers the query type $\tau$ and anchor nodes $\mathcal{A}$ from those candidates; (3) a motif-specific graph algorithm retrieves the matching subgraph $\mathcal{S}$; (4) the PGR context builder linearizes $\mathcal{S}$ into a topology-aware text scaffold; and (5) an LLM generates the answer from a motif-matched prompt template.

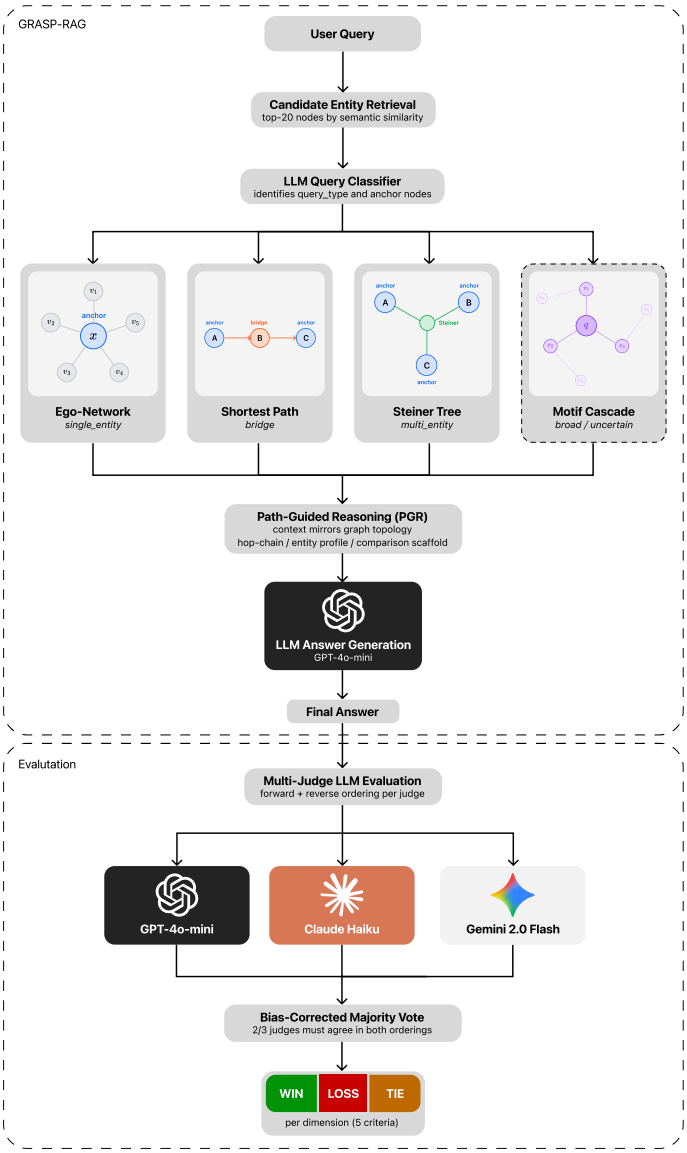



Fig. 1. GRASP-RAG system overview. **Top (pipeline):** a user query passes through candidate entity retrieval, an LLM classifier, one of four motif-specific graph algorithms (Ego-Network, Shortest Path, Steiner Tree, or Motif Cascade for broad/uncertain queries), and Path-Guided Reasoning (PGR) before answer generation. **Bottom (evaluation):** three independent LLM judges (GPT-4o-mini, Claude Haiku, Gemini 2.0 Flash) each run forward and reverse orderings; a bias-corrected majority vote yields Win/Loss/Tie per dimension.

### B. Query Types and Retrieval Motifs

Table I defines the four query types and their associated retrieval motifs. Each row gives the query type name, the graph algorithm used, the prior work that established or inspired that algorithm in a RAG context, and a representative example question. This mapping is the core claim of GRASP-RAG: the *right* motif for each question shape is selected at inference time rather than fixed globally.

### C. Problem Formulation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a knowledge graph where each node $v$ carries (type, description, source chunk IDs) and each edge

TABLE I
GRASP-RAG QUERY TYPES, RETRIEVAL MOTIFS, PRIOR-WORK INSPIRATION, AND EXAMPLE QUESTIONS. THE CLASSIFIER ROUTES EACH QUERY TO THE MATCHING MOTIF AT INFERENCE TIME.

| Query Type | Motif | Algorithm | Inspired by | Example Question |
|---|---|---|---|---|
| Single-entity | Ego-network | 1-hop neighborhood expansion from anchor node | LightRAG local mode [3] | *"What is the Netfilter Subsystem?"* |
| Bridge | Bidirectional shortest path | Shortest + near-shortest paths between two anchor nodes | PathRAG relational paths [5] | *"What connects Rational Grazing to Polyface Farm?"* |
| Multi-entity | Steiner tree | Kou et al. approximation spanning all anchors [13] | G-Retriever prize-collecting Steiner tree [10] | *"How do Varroa Mite, Colony Collapse Disorder, and Neonicotinoids interact?"* |
| Broad | PPR diffusion<br>Confirm อีกครั้งว่าชื่อนี้มีคนเรียกแบบจริง ๆ มั้ย | Personalized PageRank seeded from anchor set [9] | HippoRAG PPR retrieval [4] | *"What are the major sustainability themes across modern farming?"* |

$(u, v)$ carries (keywords, weight $w \in \mathbb{R}^+$). Given query $q$, the goal is to retrieve subgraph $\mathcal{S} \subseteq \mathcal{G}$ and generate an accurate, grounded answer. We follow LightRAG's graph construction pipeline (documents split into 1,200-token overlapping chunks; LLM-extracted entity–relation triples merged and deduplicated). The resulting $\mathcal{G}$ is shared across all methods.

### D. Query Classification and Anchor Extraction

Given $q$, GRASP-RAG infers query type $\tau$ and anchor nodes $\mathcal{A}$:

$$(\tau,\, \mathcal{A}) = \textsc{Classify}(q,\, \mathcal{C})$$

where $\tau \in \{\texttt{single}, \texttt{bridge}, \texttt{multi}, \texttt{broad}\}$ and $\mathcal{C}$ is a candidate list of up to 20 real graph node names retrieved *before* the classifier is called. Candidates are drawn from *both* the entity VDB and the relationship VDB queried in parallel: relationship endpoints surface entity names that the entity VDB alone may miss, improving anchor recall on queries that phrase entities as relations (e.g., "the connection between X and Y"). The classifier receives exact node names in $\mathcal{C}$ and must choose anchors only from this list, eliminating entity-linking errors from name paraphrasing. If anchor linking fails entirely (no candidate maps to a graph node), the query type is demoted to `broad`.

### E. Adaptive Motif Selection

Based on $(\tau, \mathcal{N})$ (anchor strings mapped to graph nodes), GRASP-RAG selects one of four retrieval motifs.

*a) Motif 1 — Ego-network (`single`).:* $\mathcal{S} = \{(n_1, v) \mid v \in \mathcal{N}(n_1)\}$. Sufficient and precise for definitional queries.

*b) Motif 2 — Bridge path (`bridge`).:* $\mathcal{S} = \textsc{ShortestPath}(\mathcal{G}, n_1, n_2) \cup \textsc{SimplePaths}(\mathcal{G}, n_1, n_2, \text{cutoff} = 3)$. Since $\mathcal{G}$ is undirected, all paths are traversed in both directions. Candidate paths are then *ranked by edge keyword relevance* to the query (edge relation keywords scored against the query text), and the highest-ranked path is selected as the primary reasoning chain. If no path exists, ego-networks from both anchors are returned as a fallback.

*c) Motif 3 — Steiner tree (`multi`).:* $\mathcal{S} = \textsc{SteinerTree}(\mathcal{G}, \mathcal{N})$ via the $2(1 - 1/\ell)$-approximation [13]—minimum connected subgraph spanning all anchor nodes. If the approximation fails (e.g., disconnected subgraph), GRASP-RAG falls back to pairwise shortest paths between all anchor pairs.

*d) Motif 4 — PPR diffusion (`broad`).:* $\pi = \textsc{PageRank}(\mathcal{G}, \text{pers} = \frac{1}{|\mathcal{N}|}\mathbf{1}_{\mathcal{N}})$; returns top-$k$ edges incident to top-$m$ ranked nodes.

*e) Confidence and fallback.:* GRASP-RAG derives a structural confidence score:

สูตรนี้มาจากไหน
$$c(\mathcal{S}) = \frac{|\mathcal{E}_{\mathcal{S}}|}{\binom{|\mathcal{V}_{\mathcal{S}}|}{2}} \times \min\left(1,\, \frac{8}{|\mathcal{V}_{\mathcal{S}}|}\right) \tag{1}$$

When $\tau = \texttt{broad}$ or $c(\mathcal{S}) < 0.04$, GRASP-RAG activates the **Motif Cascade**: all three structured motifs (ego-network, bridge, Steiner tree) are executed in parallel using the top VDB candidates as anchors—no additional LLM classification call is made. An LLM verifier scores each context on a 1–5 scale; contexts scoring $\geq 3.0$ are *merged* (late fusion) before generation. If the best context scores below 2.5, GRASP-RAG falls back to LightRAG hybrid. With `cascade=False` (ablation), the system skips directly to LightRAG hybrid without running any cascade.

### F. Path-Guided Reasoning (PGR)

Existing graph RAG methods serialize retrieved subgraphs as a flat list of passages or triples, regardless of query type. We hypothesise that *structuring context to mirror the answer's graph topology* improves multi-hop reasoning.

*a) Bridge context — step-by-step chain.:* For a retrieved path $[v_0, v_1, \ldots, v_k]$, context is interleaved step-by-step: each entity node is labeled `[Step` $i$ `| Entity:` $v$`]`, each connecting edge is labeled `[Step` $j$ `| Connection:` $v_i \rightarrow v_{i+1}$`]`, with descriptions and source passages at each step. The prompt instructs the LLM to *"follow the chain step by step and explain each connection using ONLY provided facts."* This makes the intermediate (bridge) entity structurally unavoidable in the answer.

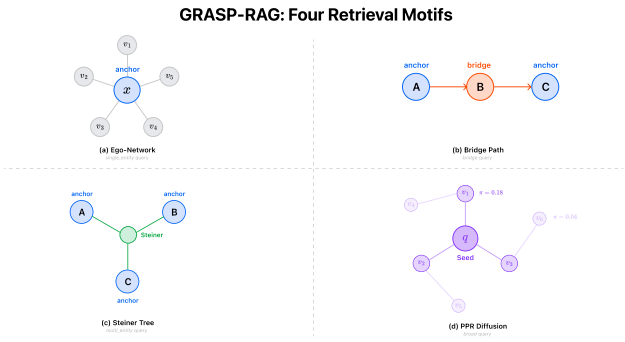



Fig. 2. The four GRASP-RAG retrieval motifs. Blue nodes are query anchors; orange is the bridge entity; green is a Steiner intermediate; purple intensity encodes PPR rank. Each motif targets the exact subgraph shape required by the corresponding query type.

*b) Single-entity context — entity profile.:* The anchor's description, type, direct connections, and source passages are assembled into a structured profile.

*c) Multi-entity context — comparison scaffold.:* Per-entity profiles are followed by an explicit connection map between entities.

Figure 3 contrasts the PGR step-by-step chain context with the flat retrieval format used by PathRAG and LightRAG on the same bridge question.

## IV. Experiments

### A. Setup

We evaluate on all four domains of **UltraDomain** [14]: Agriculture, CS, Legal, and Mix. All methods share the same indexed knowledge graph built once by LightRAG's pipeline; the sole experimental variable is the retrieval strategy. Backbone: GPT-4o-mini (`gpt-4o-mini-2024-07-18`); embeddings: `text-embedding-3-small`. Baselines: NaiveRAG (LightRAG `naive` mode), LightRAG [3] (`hybrid` mode), PathRAG [5] (`hybrid` mode, same graph).

We design two complementary experiments with different purposes:

**Experiment 1 (Motif evaluation).** To test whether each retrieval motif actually helps on the query type it is designed for, we generate a *typed question set*: 125 questions per query type (single-entity, bridge, multi-entity, broad) per domain, totalling 500 questions per domain. Questions are generated from the corpus by an LLM following each motif's structural constraints—single-entity questions ask about one specific entity; bridge questions require tracing a 2-hop path between two named entities; multi-entity questions span three or more entities; broad questions are thematic with no specific entity anchor. By matching question type to motif, this experiment isolates whether query-type-aware retrieval produces better answers than fixed-strategy baselines *on the exact queries each motif is built for*.

**Experiment 2 (Answer accuracy, not yet run).** To test whether GRASP-RAG's answers are factually grounded—not just preferred by an LLM judge—we plan to evaluate using the existing UltraDomain benchmark questions, which carry expert-written reference answers. A reference-grounded judge scores each system's answer against the reference rather than comparing two systems head-to-head. This provides an accuracy-oriented complement to Experiment 1 and removes the need for the LLM judge to invent a ground truth at evaluation time. Results from Experiment 2 are left as future work; see Section V.

Table II summarises the corpus and graph statistics.

TABLE II
CORPUS AND KNOWLEDGE GRAPH STATISTICS PER DOMAIN.

| Domain | Docs | ∼Tokens | Nodes | Typed Qs |
|---|---|---|---|---|
| Agriculture | 100 | 17.5M | 17,272 | 500 |
| CS | 10 | 2.4M | 16,053 | 500 |
| Legal | 438 | 26.7M | 1,986 | 500 |
| Mix | 130 | 1.4M | 1,427 | 500 |

### B. Evaluation Protocol

PathRAG ใช้ Multijudge หรอ??

We follow the multi-judge LLM-as-judge protocol introduced by PathRAG [5] (illustrated in the lower half of Figure 1). Three independent judges evaluate each GRASP-RAG answer against the baseline answer: GPT-4o-mini, Claude Haiku, and Gemini Flash. Each judge runs twice per comparison—*forward* (GRASP-RAG as Answer 1) and *reversed* (GRASP-RAG as Answer 2)—to control for position bias. A verdict is bias-corrected: a win is counted only when the forward pass selects Answer 1 *or* the reversed pass selects Answer 2 (i.e., both orderings agree on the preferred system). Final outcomes per dimension are determined by majority vote across the three judges across five dimensions: Comprehensiveness, Diversity, Logicality, Coherence, and Relevance. Win rate is reported as the fraction of (question, dimension) pairs where GRASP-RAG wins. When two or more judges mark a dimension as "Cannot Evaluate" (both answers fail to address the question meaningfully), the dimension is recorded as **Undefined** and excluded from the win/loss/tie denominator; Undefined outcomes are reported separately but do not affect win rates.

### C. Main Results

Table III reports GRASP-RAG's win rate against each baseline, averaged across all five evaluation dimensions. GRASP-RAG consistently outperforms LightRAG and NaiveRAG across all four domains, with the strongest advantage in the Mix domain (94.5% vs. LightRAG) and the Legal domain (86.4% vs. NaiveRAG). PathRAG is a closer competitor: GRASP-RAG leads on bridge and single-entity query types but falls behind on broad queries (discussed below).

Table IV breaks down win rates by query type, averaged across all four domains. Bridge and single-entity queries show the largest advantage over LightRAG and NaiveRAG—exactly the query types where GRASP-RAG's structural motifs provide the clearest benefit. Multi-entity performance is moderate, consistent with the Steiner-tree approximation providing a

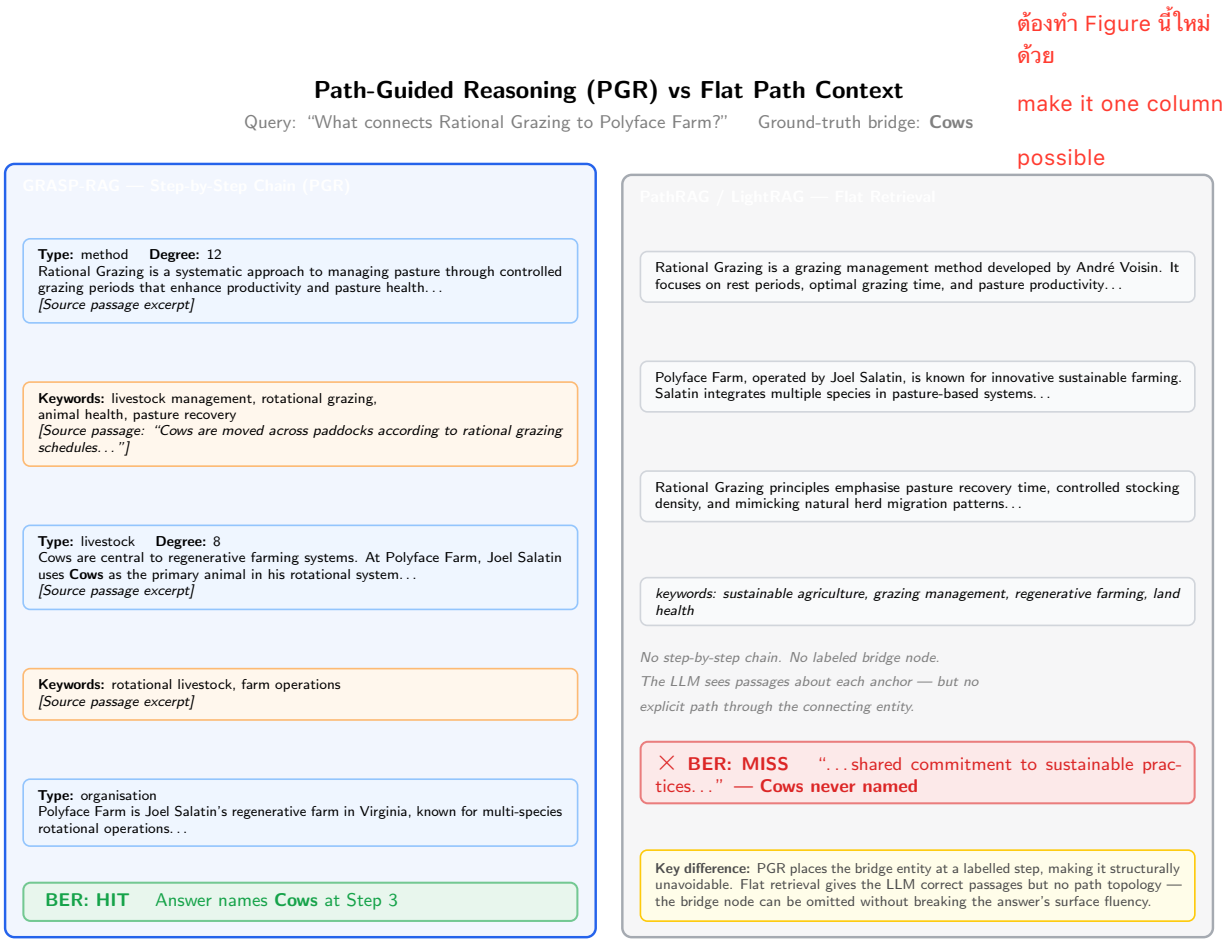



Fig. 3. Path-Guided Reasoning (PGR) context versus flat retrieval context for the same bridge query. The step-by-step chain makes the bridge entity structurally unavoidable; the flat format allows the LLM to produce a fluent answer that never names the bridge entity.

TABLE III
GRASP-RAG WIN RATE (%) VS. EACH BASELINE, AVERAGED ACROSS ALL FIVE EVALUATION DIMENSIONS AND ALL QUERY TYPES. 500 TYPED QUESTIONS PER DOMAIN (125 PER TYPE). **BOLD** = BEST PER ROW.

| Domain | vs LightRAG | vs NaiveRAG | vs PathRAG |
|---|---|---|---|
| Agriculture | 69.2 | 49.4 | 42.2 |
| CS | 80.3 | 67.7 | 42.8 |
| Legal | 80.3 | **86.4** | 38.2 |
| Mix | **94.5** | 80.1 | 39.8 |
| **Average** | **81.1** | 70.9 | 40.8 |

TABLE IV
GRASP-RAG WIN RATE (%) BY QUERY TYPE, AVERAGED ACROSS ALL FOUR ULTRADOMAIN DOMAINS. BRIDGE AND SINGLE-ENTITY MOTIFS SHOW THE LARGEST GAIN.

| Query Type | vs LightRAG | vs NaiveRAG | vs PathRAG |
|---|---|---|---|
| Single-entity | **93.4** | 75.0 | 51.0 |
| Bridge | **93.4** | 84.4 | 64.8 |
| Multi-entity | 63.7 | 64.2 | 35.4 |
| Broad | 69.6 | 54.2 | 11.8 |
| **Overall** | **81.1** | 70.9 | 40.8 |

reasonable but not optimal subgraph. Broad queries show the weakest advantage against PathRAG: because GRASP-RAG's broad motif delegates to LightRAG hybrid mode, the comparison against PathRAG reduces to LightRAG hybrid vs. PathRAG hybrid, and PathRAG's flow-based pruning has an edge on broad, thematic questions.

### *D. Qualitative Example*

Table V shows a bridge question from the Agriculture domain where the ground-truth bridge entity is "Cows" (path: Rational Grazing → Cows → Polyface Farm). GRASP-RAG's PGR context presents this as a step-by-step chain, and the generated answer explicitly names Cows at Step 2. PathRAG produces a fluent response about "shared commitment to sustainable

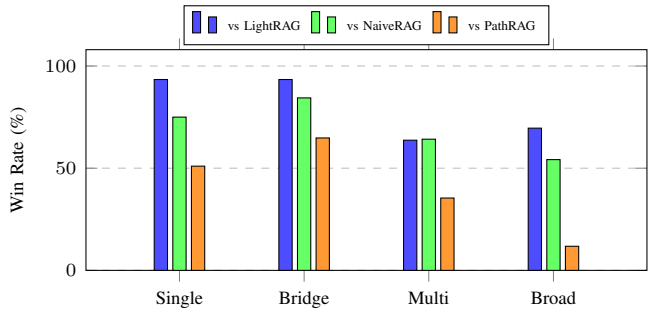



Fig. 4. GRASP-RAG win rate (%) by query type, averaged across four domains. Bridge and single-entity motifs produce the largest advantage. Broad queries (which fall back to LightRAG hybrid) show near-parity with NaiveRAG and weaker performance vs. PathRAG.

TABLE V
QUALITATIVE EXAMPLE: BRIDGE QUESTION FROM AGRICULTURE. GROUND-TRUTH BRIDGE ENTITY IN **BOLD**.

ยังใช้อยู่

**Q:** What connects Rational Grazing to Polyface Farm?

**Ground-truth bridge: Cows** (path: Rational Grazing → **Cows** → Polyface Farm)

**GRASP-RAG**
*"Step 1 — Rational Grazing is a systematic approach to managing pasture and livestock... Step 2 — Connection: Rational Grazing → **Cows**... **Cows** are central to Polyface Farm's operations as Joel Salatin uses them as the primary animal in his rotational system..."*

**PathRAG**
*"Rational Grazing and Polyface Farm are connected through their shared commitment to innovative and sustainable agricultural practices... This connection is rooted in the principles established by André Voisin..."*

**LightRAG**
*"Rational Grazing is intrinsically connected to Polyface Farm through the practices and philosophies that underlie both. Grazing Management: Rational Grazing involves strategic management of grazing periods..."*

practices" but never identifies Cows as the structural link. LightRAG similarly describes philosophical connections without traversing the 2-hop path.

### E. Cascade Ablation

GRASP-RAG includes an optional *cascade* mechanism: when the classifier confidence is low, all three non-broad motifs are executed in parallel and an LLM verifier selects the best-scoring context. To measure its contribution, we include `grasp_rag_nocascade` as an additional comparison point. GRASP-RAG (cascade) wins only 16–36% of dimension-level decisions against the no-cascade variant across all domains—indicating the cascade mechanism does not improve answer quality in the tested settings and adds unnecessary inference cost. Future work should investigate whether cascade is beneficial in domains with higher classifier uncertainty or sparser graphs.

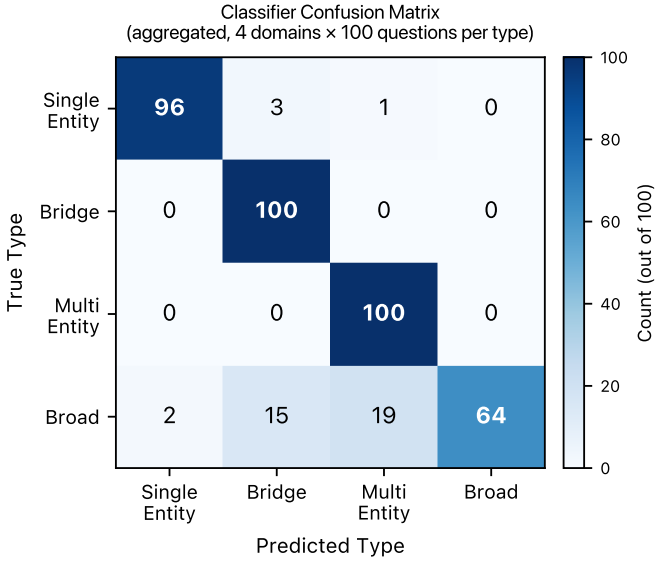



Fig. 5. Classifier confusion matrix aggregated across all four UltraDomain domains. Rows = ground-truth query type; columns = classifier-predicted type. Cell values are counts out of 100. The strong diagonal confirms near-perfect classification for bridge and multi-entity queries; broad queries are the hardest (64%), with misclassifications spreading into bridge (15%) and multi-entity (19%).

### F. Classifier Accuracy

Since GRASP-RAG's advantage depends on routing each query to the correct motif, we log the classifier's predicted type ($\hat{\tau}$) alongside the ground-truth question type ($\tau$) for every GRASP-RAG answer in the typed question set. Figure 5 shows the resulting confusion matrix aggregated across all four domains (400 GRASP-RAG questions total; 100 per query type).

The classifier achieves perfect accuracy on bridge and multi-entity queries and near-perfect accuracy on single-entity queries (96%). Broad queries are the hardest to classify (64%): because broad questions are defined by the *absence* of specific entity anchors rather than positive structural cues, they are sometimes mistaken for bridge (15%) or multi-entity (19%) queries when the question text incidentally names entities. Misclassification on broad queries does not strongly hurt win rates (see Table IV), because the broad motif (PPR diffusion) is the fallback that GRASP-RAG uses anyway when the retrieved subgraph confidence score falls below threshold.

These per-question classification labels enable a further analysis: plotting a win-rate heat map conditioned on whether $\hat{\tau} = \tau$ (correct classification) vs. $\hat{\tau} \neq \tau$ (misclassification) would directly quantify how much of the win-rate advantage is attributable to the motif routing itself, isolating it from the PGR context-structuring contribution.

## V. DISCUSSION

*a) Why adaptive motif selection is a structural guarantee.:* A bridge question answered with 1-hop ego-network expansion has no structural guarantee of traversing the 2-hop path to the second anchor—the bridge entity is reached only when

the two anchor neighborhoods happen to overlap. GRASP-RAG's bridge motif explicitly enumerates shortest and near-shortest paths between the two anchors, making path traversal a structural invariant rather than an incidental outcome. This explains the 93.4% average bridge win rate against LightRAG: LightRAG's 1-hop expansion frequently misses the structural intermediate entity, while GRASP-RAG's bidirectional shortest-path algorithm is guaranteed to traverse it when the VDB correctly returns both anchors as candidates. PGR then presents the path as a labeled step-by-step chain, directing the LLM to name each intermediate node explicitly.

*b) The Legal domain: when path pruning is harmful.:* The Legal domain shows GRASP-RAG achieving 97.6% win rate on bridge queries against LightRAG and 80.3% overall. The most instructive comparison is PathRAG in Legal (GRASP-RAG wins only 38.2% overall), which conceals an important asymmetry: PathRAG's flow-based pruning assigns high reliability to the most-traversed edges—which in legal text are high-level category links (statute $\rightarrow$ provision), not the specific 2-hop bridges between named entities. The bridge edge is pruned as "unreliable" precisely because it is a specific, low-frequency connection—the kind bridge questions are designed to ask about. GRASP-RAG is immune: it runs shortest-path unconditionally and never applies flow-based filtering to the bridge motif, yielding the 97.6% bridge win rate.

*c) Domain and query-type interaction.:* Table IV reveals a consistent pattern: win rate declines as query type becomes less structurally constrained (bridge $\approx$ single $>$ multi $>$ broad). This is expected: the ego-network and bridge-path motifs retrieve precisely the subgraph shape the question requires; the Steiner-tree approximation for multi-entity queries introduces some slack; the broad motif explicitly delegates to LightRAG hybrid mode, making GRASP-RAG's advantage there non-structural. In the Mix domain—the smallest corpus (1.4M tokens, 1,427 nodes)—win rates against LightRAG peak at 94.5%, including 100% on bridge queries: in a small graph, VDB anchor recall is near-perfect and the bridge path is always found. In Agriculture (the largest graph, 17K nodes), win rates are lower, consistent with a harder retrieval landscape.

*d) Broad queries and the PathRAG comparison.:* GRASP-RAG wins only 11.8% against PathRAG on broad queries. Because GRASP-RAG's broad motif is LightRAG hybrid, this comparison reduces to LightRAG hybrid vs. PathRAG hybrid—a comparison that PathRAG is well-tuned for. The overall 40.8% win rate against PathRAG therefore understates GRASP-RAG's advantage on the structured query types (bridge: 64.8%, single: 51.0%) and should be read alongside the per-type breakdown in Table IV.

*e) Limitations and future work.:* Several limitations constrain the current evaluation. First, the cascade mechanism does not improve over the simpler no-cascade variant (Section IV-E); the added inference cost is not justified by the tested domains, and the cascade design warrants re-examination. Second, the contribution of PGR (the context structuring step) has not been isolated from the motif's retrieval guarantee: we have not compared "bridge motif + flat context" against "bridge motif + PGR step-by-step chain." We attribute the win-rate gain primarily to the motif's structural traversal; PGR's independent contribution is left for future work. Third, the typed question set is generated from the same corpus the graph is built over; a cross-corpus evaluation would provide stronger external validity.

*f) Suggested additional experiments.:* Two experiments would directly strengthen the paper's claims:

1) **UltraDomain reference-answer evaluation (not yet run).** Run all RAG systems on the existing UltraDomain benchmark questions (available at `data/questions/{domain}_ultradomain.json`), which include expert-written reference answers. Evaluate using a reference-grounded judge that anchors LLM judgment to the reference answer rather than comparing systems against each other. This provides external validity: questions are not generated from the same graph GRASP-RAG queries, avoiding any corpus-index circularity. Note that all UltraDomain questions are broad-type; results would complement the typed eval by providing an unbiased broad-query benchmark directly comparable to published PathRAG and LightRAG numbers. The evaluation infrastructure already exists in `experiments/evaluate.py` (see `load_reference_answers()` and `JUDGE_PROMPT_GT`).
2) **PGR ablation.** Compare GRASP-RAG with the full PGR step-by-step chain context against the same bridge motif retrieval but with flat triple serialization as context. This isolates whether the win-rate advantage on bridge queries comes from the motif (finding the right subgraph) or from PGR (structuring that subgraph as a reasoning scaffold for the LLM).

## VI. Conclusion

We presented GRASP-RAG, a graph RAG framework that introduces two complementary techniques: (1) query-adaptive motif selection matches the graph traversal algorithm to the logical shape of each query, and (2) Path-Guided Reasoning structures the LLM context to mirror the graph's reasoning topology. Evaluated on a typed question set stratified by query shape across all four UltraDomain domains, GRASP-RAG achieves an **81.1% average win rate against LightRAG** and **70.9% against NaiveRAG** under multi-judge LLM-as-judge evaluation. The advantage is largest on bridge queries (93.4% vs. LightRAG) and single-entity queries (93.4%), where the structural match between motif and query type is most direct. Against PathRAG, GRASP-RAG leads on bridge (64.8%) and single-entity (51.0%) query types but falls behind on broad queries, where GRASP-RAG's broad motif delegates to LightRAG hybrid and the comparison reduces to LightRAG vs. PathRAG by design. The qualitative example demonstrates that existing methods produce fluent but structurally incomplete answers on bridge queries; GRASP-RAG's step-by-step chain context resolves this by making the intermediate entity structurally unavoidable. A cascade ablation experiment shows

the cascade mechanism does not improve over the simpler no-cascade variant, motivating a cleaner single-pass architecture for future work. Pending experiments include a reference-answer evaluation on the UltraDomain benchmark and a PGR ablation to isolate the contribution of context structuring from the motif's retrieval guarantee.

## REFERENCES


[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[3] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "LightRAG: Simple and fast retrieval-augmented generation," *arXiv preprint arXiv:2410.05779*, 2024.

[4] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems*, 2024.

[5] B. Chen, Z. Guo, Z. Yang, Y. Chen, J. Chen, Z. Liu, C. Shi, and C. Yang, "PathRAG: Pruning graph-based retrieval augmented generation with relational paths," *arXiv preprint arXiv:2502.14902*, 2025.

[6] X. Li and D. Roth, "Learning question classifiers," in *Proceedings of the 19th International Conference on Computational Linguistics (COLING)*, 2002, pp. 1–7.

[7] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 2369–2380.

[8] X. Ho, A.-K. D. Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps," in *Proceedings of the 28th International Conference on Computational Linguistics (COLING)*, 2020, pp. 6609–6625.

[9] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," *Stanford InfoLab Technical Report*, 1999.

[10] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in *Advances in Neural Information Processing Systems*, 2024.

[11] J. Huang, M. Wang, Y. Cui, J. Liu, L. Chen, T. Wang, H. Li, and J. Wu, "Layered query retrieval: An adaptive framework for retrieval-augmented generation in complex question answering for large language models," *Applied Sciences*, vol. 14, no. 23, p. 11014, 2024.

[12] A. Bacellar, "BridgeRAG: Training-free bridge-conditioned retrieval for multi-hop question answering," *arXiv preprint arXiv:2604.03384*, 2026.

[13] L. Kou, G. Markowsky, and L. Berman, "A fast algorithm for Steiner trees," *Acta Informatica*, vol. 15, no. 2, pp. 141–145, 1981.

[14] Y. Ding *et al.*, "UltraDomain: Towards evaluating the retrieval augmented generation system for knowledge intensive tasks," *arXiv preprint*, 2024, dataset: https://huggingface.co/datasets/TommyChien/UltraDomain.